Line Following Robot Project

Group 03 A: Anton Hawkins, Cody Carter, Khush Patel, James Buckland-Blair

ENMT221 – Mechatronics Design 1

2026-S2

9 October 2026

University of Canterbury

Abstract

This report details the design, assembly and testing of an autonomous line following robot that navigates a white line on a black surface under strict power and component constraints. The chassis is a printed circuit board carrying a separate, adjustable sensor board, which reduced mass and increased stiffness. Two MM28 motors, chosen for their stall torque, drive the wheels through a single-stage friction-wheel transmission with an effective reduction of 18.5:1 (50 mm wheel on a tape-wrapped shaft of a 2.7 mm diameter). The transmission went through several iterations to reduce slip. Five TCRT5000 sensors on the sensor board produce analogue voltages proportional to surface reflectivity; sensor-to-sensor variation required individual calibration. A closed-loop proportional controller takes the position of the strongest calibrated sensor as its error and applies a differential PWM correction about a position-dependent base speed; derivative action was trialled and removed because it gave no consistent improvement. The robot completed all three demonstration runs, with a best time of 17.68 s and placing 12th in the cohort. In reflection, a gear-based transmission would remove the slip inherent in friction drive, and replacing the 0.7 V diode between the battery and regulator with a lower-drop device would improve brown-out margin. Once the best sensor position was found, integrating the sensor and main boards would also reduce size and inertia.

Table of Contents

1 Introduction 1
2 Design Specifications 2
2.1 Constraints and Functional Requirements 2
2.2 Design Decisions and Rationale 3
2.2.1 Microcontroller 3
2.2.2 PCB Architecture 4
2.2.3 H bridge motor control 4
2.2.4 Motor 5
3 Concept Evaluation and Group Consensus 7
3.1.1 Alternate decision 1: Two-Stage Belt and Pulley Configuration 8
3.1.2 Alternate decision 2: Two-Stage Spur Gear Drive 9
4 Mechanical Design 10
4.1 Materials 10
4.1.1 Chassis Selection 10
4.1.2 3D-Printed PLA Components 10
4.2 Mechanical Concept 10
4.2.1 Motion transmission 10
4.2.2 Chassis and sensor array 12
4.2.3 Battery housing 12
4.2.4 Wheel and motor mounting 12
4.3 Mechanical design and iterations 13
4.3.1 Motor housing and wheel iterations 13
4.3.2 Wheel Traction iterations 15
4.3.3 Battery mount and chassis iterations 17
5 Electronics 19
5.1 PCB Design 19
5.1.1 PCB board layout 19

5.1.2 Sensor board 19
5.1.3 Main board 19
5.2 PCB Iterations and troubleshooting issues 19
5.3 Bluetooth Programming 20
6 Computation 21
6.1 Sensor Reading and Calibration 22
6.2 Line Position Detection 23
6.3 Motor control 23
6.4 Development of the Control Algorithm 24
7 Comparison between other groups 29
8 Results 30
8.1 Performance on the demonstration course 30
9 Reflection 31
10 Conclusion 33
11 References 34
12 Appendix 35
12.1 Appendix A 35
12.2 Appendix B 38

1 Introduction

Autonomous line following robots provide a practical application of mechatronics by integrating mechanical systems, electronics, sensing and embedded control into a single autonomous platform. This report documents the design, fabrication, and testing of a robot that can autonomously follow a defined track while meeting the dimensional and power constraints specified in the project brief.

The primary objective of this project is to develop a robot that can reliably follow a white line on a black surface through straight sections and curves of varying radius. A secondary objective is to minimise course completion time by operating at the highest practical speed while maintaining accurate and stable line tracking. This requires integrating an optical sensing system, embedded control, differential motor drive, and mechanical transmission into a coordinated closed-loop system. The design process therefore focuses on balancing high-speed tracking responsiveness with dynamic stability, overall power efficiency, and physical constraints.

Figure 1 illustrates the robot's functional structure. Five TCRT5000 sensors emit infrared light and generate analogue voltages proportional to surface reflectivity. The ATmega4808-AFR microcontroller reads these voltages using its on-chip ADC, executes a proportional control algorithm, and outputs two PWM signals. These signals drive low-side N-channel MOSFET stages that control the two MM28 motors through the friction-wheel mechanical transmission. Four AA NiMH cells connected in series provide a nominal supply voltage of 4.8 V, while a 3.3 V linear regulator supplies the logic circuitry.

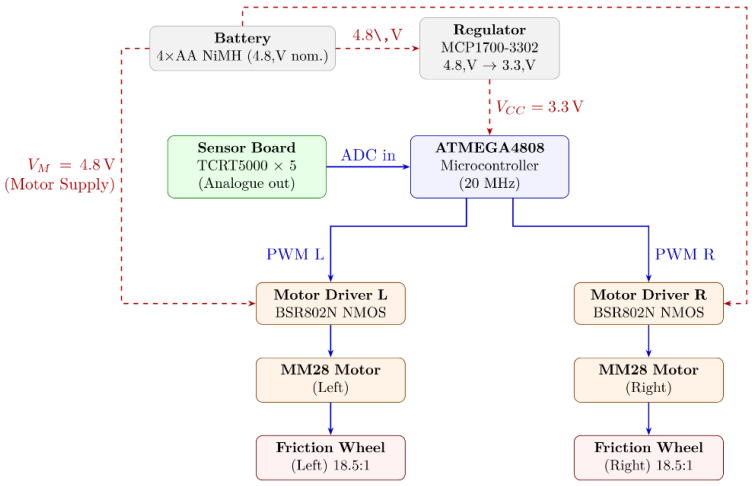


Figure 1: Functional Model for the line following robot.

2 Design Specifications

2.1 Constraints and Functional Requirements

The hard constraints imposed by the project brief [1] guided all design decisions summarised in Table 1.

Table 1: Project constraints imposed by the project brief.

Parameter	Requirement
Microcontroller	One ATmega328-PU or ATmega4808-AFR
Motors	Maximum four; MM10 or MM28
Sensors	Maximum five; TCRT5000 reflective infrared optical sensor
Power	Four AA NiMH rechargeable batteries
PCB dimension	JLCPCB-compatible, not exceeding 100 mm
3D-printed PLA	Maximum 100 g
Sheet material	4.75 mm MDF within a 200 mm × 200 mm bounding box
Shafts / bearings	3 mm steel shaft; MR63ZZ miniature bearings
Tyres / drive elements	Rubber bands, O-rings, bicycle inner tube, or thread
Course area	Within 2.4 m × 1.2 m
Line width	15-20 mm
Minimum radius	80 mm to line centre
Electronic components	List in Appendix A of project brief

From the brief constraints and desired objective of minimising course time, the following functional requirements were derived:

- The robot must detect the boundary between the white line and the black surface using five TCRT5000 sensors mounted on a separate sensor board, distributed across the robot's width, with analogue signals read by the ATmega4808 microcontroller.
- The robot should follow the line using a closed-loop controller with a minimum control-loop rate of 2000Hz to enable response to corners of 80 mm minimum radius at the target speed.

- The robot should regulate motor speed independently on the left and right channels via PWM to produce differential steering.
- The PCB should provide UPDI programming access and a UART bootloader interface compatible with the ENEL198 cable.
- The power supply should maintain the 3.3 V rail within the MCP1700 regulation band during motor inrush transients.

2.2 Design Decisions and Rationale

The following sub-sections document the key design decisions made by the group, with supporting analysis and rationale for each choice.

2.2.1 Microcontroller

The ATmega4808-AFR was selected over the ATmega328-PU. The decisive advantages, proven by the datasheets:

Internal oscillator accuracy: The ATmega4808's OSC20M oscillator is factory-calibrated to ±1.5% at room temperature (25°C) and ±2.0% across 0°C-70°C. The chip stores factory-measured frequency error values in its Signature Row (SIGROW), which the software can use to compensate the UART baud rate register automatically. This accuracy meets the ~2.5% tolerance threshold required for reliable 115200 baud UART serial communication without an external crystal [2]. For the ATmega328, the factory calibration for the internal 8 MHz RC oscillator is only specified to ±10% at 25°C [3]. A ±10% clock variation can cause framing errors during high-speed UART communication, so reliable serial data transmission requires an external crystal unless you implement custom runtime recalibration.

ADC input count: The ATmega4808 provides 12 external ADC channels. Specifically, Port D has 8 channels (PD0-PD7) and Port F has 4 channels (PF2-PF5). This provides dedicated analogue pins for 5-line sensors plus battery monitoring (if needed) without requiring external analogue multiplexers [2, 4]. By comparison, the 28-pin DIP ATmega328 offers only 6 ADC channels on external pins (PC0-PC5), as that package omits ADC6 and ADC7 [3].

UPDI Programming: The ATmega4808 uses a single-wire UPDI interface on a dedicated pin (pin 27) [4], avoiding conflicts with general-purpose peripheral pins. The ATmega328 relies on SPI/ISP programming across pins PB3-PB5, where PB3 functions as both MOSI and the OC2A PWM output channel [3, 4].

PORTMUX Flexibility: On the ATmega4808, default pin allocations place EXTCLK, USART0 TX, and TCA0 WO0 together on pin PA0. The internal PORTMUX controller allows remapping USART0 (via USARTROUTEA) and TCA waveform outputs (via TCAROUTEA) to alternate pin groups without requiring PCB changes [4].

Flash Memory: The ATmega4808 provides 48 kB of flash against 32 kB on the ATmega328 [1], sufficient for the calibration, control and recovery routines and for further development.

2.2.2 PCB Architecture

The group designed a two-PCB system with a main board and a sensor board. The main board uses the ATmega4808, MCP1700 power supply, two BSR802N motor-driver stages, UPDI and UART programming headers, and all power-management circuitry.

The sensor board is separate for adjustability; placing both boards on a single board would fix the distance between the microcontroller and the sensor array.

2.2.3 H bridge motor control

An H bridge has several advantages for this robot's line following application. It can change motor direction instantly and provide braking when required. This allows precise control in corners and the ability to pivot at high speeds.

The H-bridge was not implemented due to complexity and reliability concerns. After consulting with 3rd-year Mechatronics students, we found the H-bridge to be inconsistent. It also introduces a shorting risk: simultaneous conduction of both switches on one side of the bridge due to a switching delay would create a direct short across the supply rail. Avoiding this would require complex gate-drive timing, which was beyond the scope of this project and timeframe.

The low-side MOSFET drive limits each motor to forward motion or coasting, with no reverse or active braking capability. Differential steering relies on slowing the inner wheel to zero speed rather than reversing its direction. This reduces the robot’s ability to pivot in tight corners such as the hairpin discussed in Section 9.

2.2.4 Motor

The Multicomp MM28 was selected over the MM10 based on an analysis of the motor datasheets (Table 2) and a weighted decision matrix across multiple figures of merit (Table 3).

Table 2: Motor parameter comparison (MM10 vs MM28, from supplier datasheets).

Parameter	MM10	MM28	Preferred
Mass (g)	17	42	MM10
Rated voltage range (V)	1.5 – 3	3 – 6	MM28
No-load speed (rpm)	16 300	9 600	MM10
No-load current (A)	0.38	0.22	MM28
Stall torque (N·m)	0.0044	0.0112	MM28
Max-efficiency torque (N·m)	0.001	0.002	MM28
Max-efficiency current (A)	1.10	0.99	MM28
Max-efficiency speed (rpm)	12 000	8 000	MM10
Max efficiency (%)	35.6	55.2	MM28

The motor selection was evaluated using the weighted decision matrix presented in Table 3, which compares the MM10 and MM28 motors across the selected performance criteria.

Table 3: Motor decision matrix (scores 1-5).

Figure of merit	Weight	MM10	MM28
Weight	0.15	4	3
Torque	0.30	2	5
Current	0.10	3	4
Speed	0.25	5	3
Voltage	0.05	3	4
Efficiency	0.15	2	5
Weighted score (/5)	1.00	3.2	4.05
Rank		2	1 (Selected)

The MM28 was selected over the MM10 based on four considerations. First, the MM28 has a rated voltage range of 3-6 V, which directly encompasses the nominal 4.8 V battery supply. The MM10 is rated for 1.5-3 V. Although the MM10 could potentially operate at a 4.8 V supply using PWM, a maximum duty cycle of approximately 62% would be required to limit the average applied voltage to 3 V. This would constrain available motor speed and impose an additional limitation on the motor-control hardware. The electrical protection requirements associated with operating the MM10 above its rated voltage were outside the scope of this analysis.

Second, the MM28 provides substantially greater torque than the MM10. The MM28 has a stall torque of 0.0112 N·m, compared with 0.0044 N·m for the MM10, an increase of approximately 2.55×. This provides greater torque margin during acceleration and high-demand manoeuvres such as cornering. At the maximum-efficiency operating point, the MM28 also provides twice the torque of the MM10 (0.002 N·m compared with 0.001 N·m).

Third, the MM28 achieves 55.2% efficiency at its optimum operating point, versus 35.6% for the MM10. The higher efficiency indicates that more electrical input power is converted into mechanical output, with correspondingly lower losses. This reduces energy consumption and motor heating during operation.

Finally, although the MM28 has a greater mass than the MM10 (approximately 2.5× heavier), this additional mass benefits the selected friction-wheel transmission. With the motor positioned above the rear drive wheels, its added weight increases the normal force on the driven wheels. This increases the maximum available frictional force and reduces wheel slip during high-torque operation. However, the increased mass also increases the robot's inertia and was therefore considered a trade-off rather than an unqualified advantage.

3 Concept Evaluation and Group Consensus

The initial individual phase introduced the design tools and analysis methods. Each team member independently designed a PCB and motor transmission and carried out a kinematic analysis of the transmission system. This phase aimed to build familiarity with PCB design in KiCad under JLCPCB 2-layer design rules, datasheet interpretation, and first-principles mechanical analysis, rather than produce a final design for direct adoption.

Following the individual design feedback, each member's work was reviewed, and the best elements were selected to combine into a single integrated design. Cody's individual design (ATmega4808) provided the foundation for the PCB, while the modular approach of separating the main board from the sensor board was retained. This modularity was chosen over a single unified board because it kept the robot's physical length and sensor extension adjustable during testing. Keeping the boards separate also prevented the mechanical dimensions from being locked, which was critical given the time constraints.

During the layout phase, the design was modified so the main PCB served as the robot's structural chassis. Several test points were added across the board to detect errors, verify power delivery, trace faults, and confirm individual component functionality without risking short circuits on the dense microcontroller pins.

For the drivetrain, friction wheels were considered because they are more efficient than a two-stage belt and pulley drive and a two-stage gear drive, weigh less, are easier to manufacture, and are easier to tune within the available build time. Appendix B shows the conceptual drawings of all three transmissions, with the friction wheel clearly presented as the simplest design to implement.

To select the optimal drivetrain, three transmission configurations were evaluated: a single-stage friction drive, a two-stage belt and pulley drive, and a two-stage gear drive. Table 4 summarises the decision matrix, comparing these candidates across key performance and manufacturing criteria.

Table 4: Motion transmission decision matrix (scores 1-5).

Evaluation Criteria	Weight	Friction Drive	Belt & Pulley	Gear Drive
Mechanical Efficiency	0.25	5	4	3
Slip Resistance & Traction	0.20	3	5	5
System Mass & Rotating Inertia	0.20	5	4	3
Tolerance to Frame Deflection	0.15	4	4	1
Shock & Vibration Absorption	0.15	5	4	1
Manufacturing & Assembly	0.05	5	3	2
Weighted score (/5)	1.00	4.45	4.15	2.75
Rank		1 (Selected)	2	3

The friction-wheel drive was selected. Its score advantage over belt-and-pulley (4.45 vs 4.15) stems primarily from higher mechanical efficiency (no belt bending losses), simpler manufacturing (no intermediate shafts, bearings, or belt tensioning), and lower mass. Although its slip-resistance score is lower (3 vs 5), this is mitigated by the MM28's downforce contribution and using a compliant rubber O-ring tyre on the wheel rim, which increases the contact normal force and improves grip. Accordingly, for any multi-stage drivetrain, the overall ratio (N_{total}) and efficiency (η_{total}) are governed by the effects of sequential reductions acting in series:

$$N_{\text{total}} = N_1 \cdot N_2 = \frac{D_{\text{driven, 1}}}{D_{\text{driver, 1}}} \cdot \frac{D_{\text{driven, 2}}}{D_{\text{driver, 2}}}$$

$$\eta_{\text{total}} = \eta_{\text{stage 1}} \cdot \eta_{\text{stage 2}}$$

Because individual stage efficiencies are fractional ($\eta < 1.0$), compounding multiple stages reduces the power output. Given the MM28 motor's torque constraints and the manufacturing tolerances of FDM 3D printing, a single-stage architecture minimised these transmission losses.

3.1.1 Alternate decision 1: Two-Stage Belt and Pulley Con⁄guration

While a belt system is efficient, a two-stage belt reduction introduces minor tracking delays from belt compliance (stretching) during rapid directional changes. The

compounded efficiency ($\eta_1 \cdot \eta_2$) restricts the net torque from the MM28 motor. For precise line following, these small delays can induce oscillations as the PID controller attempts to centre the robot over the line.

The design of the four pulleys, two belts and a shaft increases both static mass and rotating inertia. FDM manufacturing variances make it difficult to guarantee the precise shaft parallelism required to prevent the belts from walking laterally off their pulleys.

3.1.2 Alternate decision 2: Two-Stage Spur Gear Drive

The transmission efficiency is limited by raw plastic-on-plastic sliding contact. The micro-ridges and layer lines from the FDM printing profile create continuous sliding friction and micro-binding at each tooth mesh, reducing torque when multiplied across two stages.

A two-stage gear train introduces mechanical backlash (loose play) at each tooth mesh. When the robot makes rapid corrections to follow a track, this play creates a deadband where the motor rotates slightly before the wheels respond, introducing lag that destabilises the line-sensor feedback loop.

Spur gears require strict centre-to-centre distance tolerances. Minor 3D printing shrinkage or warping can shift pitch-circle alignment, altering tooth clearance and risking tight spots, micro-binding, or accelerated tooth wear.

4 Mechanical Design

4.1 Materials

4.1.1 Chassis Selection

Using a PCB as the base chassis was an obvious choice for us, as a fibreglass PCB is strong enough to support the weight of the batteries and motors. This also avoided an extra weight a 3D printed chassis would add to the robot. PLA has a yield stress of 45 MPa, compared to 450 MPa for fibreglass. This made fibreglass the obvious choice, since it could easily support the weight of the motor and batteries.

4.1.2 3D-Printed PLA Components

3D-printed PLA components were used for structural supports outside the base PCB chassis because they allow fast prototyping and production. However, these components can suffer from delamination between layers under load, as well as slight variation in printed sizes especially for small parts due to the tolerances of the 3D printer.

4.2 Mechanical Concept

4.2.1 Motion transmission

Using the selected friction wheel drive transmission, several design constraints apply, specifically the 4.8 V supply voltage. The graphs in Appendix A were used to extrapolate a no-load speed of 14,000 RPM and a stall torque of 170 g·cm, since only 3 V and 6 V values were given for the MM28 motor. Because the shaft-to-wheel efficiency was estimated at approximately 0.7, some form of tensioner or interference fit will be required to force the shaft into the wheel. Because the wheel-to-ground efficiency was estimated at around 0.85, and the robot weighs approximately 400 grams, the target speed was set at 0.5 meters per second to balance straight-line speed with the ability to take sharp corners. The MM28 motor shaft has a radius of 1mm.

Based on these parameters, a 32 mm wheel was chosen initially because smaller wheels carry less mass, which therefore improves acceleration as the inertia is lower. Adding heat shrink increases the shaft radius, which decreases the gear ratio. The gear ratio between the wheel and the motor shaft is calculated by the following equation.

$$N = \frac{r_{wheel}}{r_{shaft}} = \frac{16.0\ mm}{1.0\ mm} = 16$$

To reach the desired speed, the following equations calculate the wheel's angular speed. The efficiency of 0.85 between the ground and the wheel is then factored in, giving a final target wheel angular velocity of approximately 351 RPM.

$$\omega_{wheel} = \frac{v}{r_{wheel}} = \frac{0.5\ m/s}{0.016\ m} = 298.4$$

$$\omega_{wheel,actual} = \frac{\omega_{wheel}}{\eta_{ground}} = \frac{298.4\ RPM}{0.85} \approx 351\ RPM$$

The upper bound for the gear ratio was 60% of the motor's no-load speed. This is required for the pulse-width modulation of the motor to work correctly; the motor must be operating within the linear torque range:

$$N_{max} = \frac{0.6 \cdot \omega_{NL}}{\omega_{wheel,actual}} = \frac{0.6 \times 14000\ RPM}{351\ RPM} \approx 23.9$$

The minimum gear ratio is constrained by the motor's stall torque. The torque of the motor can be calculated using the following equation, and the gear ratio can then be calculated by dividing the torque of the wheel by the stall torque of the motor. The shaft-to-wheel efficiency needs to be factored in.

$$\tau_{wheel} = \frac{\mu \cdot m \cdot g \cdot r_{wheel}}{2} = \frac{0.65 \times 0.4\ kg \times 9.81\ m/s^2 \times 0.016\ m}{2} \approx 208\ g \cdot cm$$

$$N_{min} = \frac{\tau_{wheel}}{\tau_{stall} \cdot \eta_{shaft}} = \frac{208\ g \cdot cm}{170\ g \cdot cm \times 0.7} \approx 1.75$$

The designed nominal ratio of 16 sits safely between the minimum torque limit of 1.75 and the maximum speed limit of 23.9. This confirms the ratio is physically possible, keeps the motor running at approximately 5600 RPM within the linear control region, and provides sufficient margin even when heat shrink is added to increase the shaft radius.

$$1.75 \leq N \leq 23.9$$

4.2.2 Chassis and sensor array

Using the PCB as a base chassis, as previously mentioned, offers many benefits; the board was designed around both the electronic components and the LFR's structural requirements.

The sensor board was attached in front of the PCB using 3D mounts to position it at an optimal distance, balancing wheel reaction time while increasing inertia and improving steering control. The front structural mount also included a nub that acted as a support to position the sensors at the correct height above the ground which was 2.5mm according to the datasheet [6].

4.2.3 Battery housing

The battery housing was initially designed to sit above the PCB on a 3D-printed platform with standoffs. This custom platform was chosen because it was easy to design and fit the chassis mounting holes. It also positioned the batteries optimally, providing sufficient weight near the motors to reduce slipping between the wheel and the ground.

4.2.4 Wheel and motor mounting

The wheels and motor mount were 3D-printed, allowing quick adjustment of gear ratios. The motor mount served two purposes: holding the motor in a fixed location to prevent it from rotating within its housing and acting as a tensioner that used rubber bands to force the motor shaft into the wheel, enabling efficient friction-wheel transmission, as shown in Figure 2.



Figure 2: Original motor housing.

4.3 Mechanical design and iterations

4.3.1 Motor housing and wheel iterations

The initial motor housing prototype had issue providing enough tensioning force onto the robot's wheel. This was because the rubberbands directly contacted the motor which caused an uneven force and allowed for the rubberbands to slip and move around. The second design incorporated a hinge design which would allow for an even force across the motor as showed in Figure 3. This was based on the issues seen in the initial prototype, and a design which could fit multiple sizes of wheels to change the gear ratio.

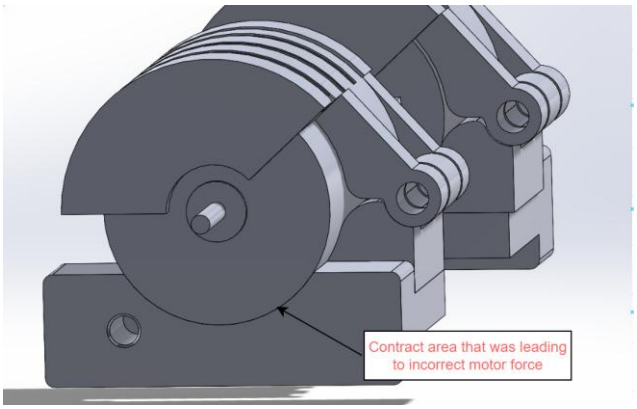


Figure 3: Diagram highlighting the contact area between motor bracket and motor.

Stall torque and speed started becoming an issue where there was not enough starting torque at the start line and the robot was travelling too fast overshooting corners. The motors required around a PWM value of 50 to overcome the stall torque were going above 60 would mean that the robot was travelling to fast to react. This meant that it was decided to change the gear ratio to allow for a larger range between optimal speed and stall torque. Increasing the wheel diameter increases the gear ratio which allows for higher torque and lower speed. This also allowed for use to operate at a higher PWM value without overshooting corners and there was also ample starting torque. A wheel diameter of 50 mm was chosen to try to use PWM value of 127 which would mean a duty cycle of 50%. This also required the motor housing to be changed to fit these new wheels.

In the second design the rubber bands provided a tensioning force by surrounding the whole motor attachment by surrounding the whole housing as seen in Figure 4. This provided an uneven force down onto the wheel as the motor was pivoting off the wheel creating a smaller contact area as the far end of the shaft lifted off the wheel. The redesigned motor fully encased the motor to allow for the shaft to contact evenly on the wheel. The rubber bands acted as an adjustable brake by changing the amount or position

of the bands. This was because adjusting the tension changed the normal force against the wheel which increased the frictional force slowing the robot down. This allowed for greater control around corners as the robot was now able to slow down quickly instead of carrying through the corners because of the inertia in the system. The increased need for power to overcome the brake was not an issue as it was operating at around 30-40 % of the full PWM.

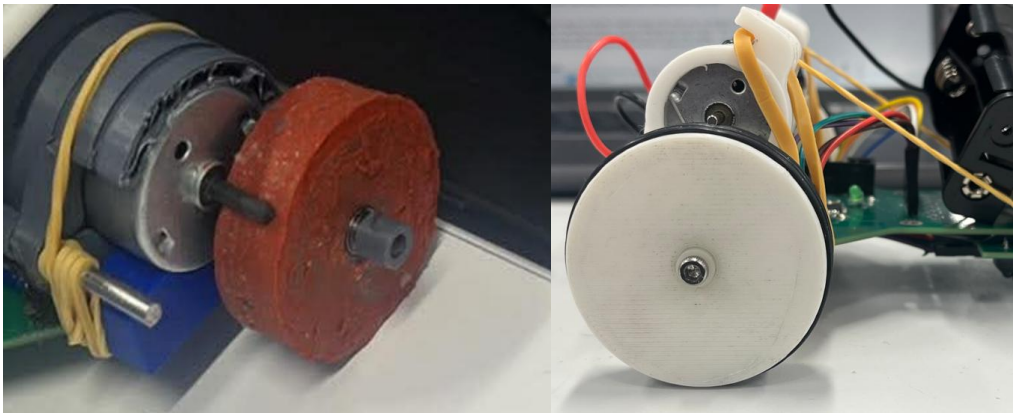


Figure 4: Image showing the suboptimal rubber band use that provide poor normal force (left) compared to current design (right) that has provides superior force through application of a moment.

After getting the robot to consistently complete the course in around 15 seconds the top speed seemed to be the limiting factor, therefore 3D printed shaft expanders were made to reduce the gear ratio and therefore increase the possible speed. This helped the robot go faster but was difficult to tune and after a few tests the robot started to brown out multiple times each run. After much troubleshooting, it was decided to revert to the old design with just the shaft which reduced the gear ratio back to the high torque low speed design. A dimensioned drawing for the final motion transmission is shown in Figure 5 this gave the robot an 18.5 gear ratio with a 50 mm wheel inclusive of the O-rings and a 2.7 mm shaft inclusive of the electrical tape.

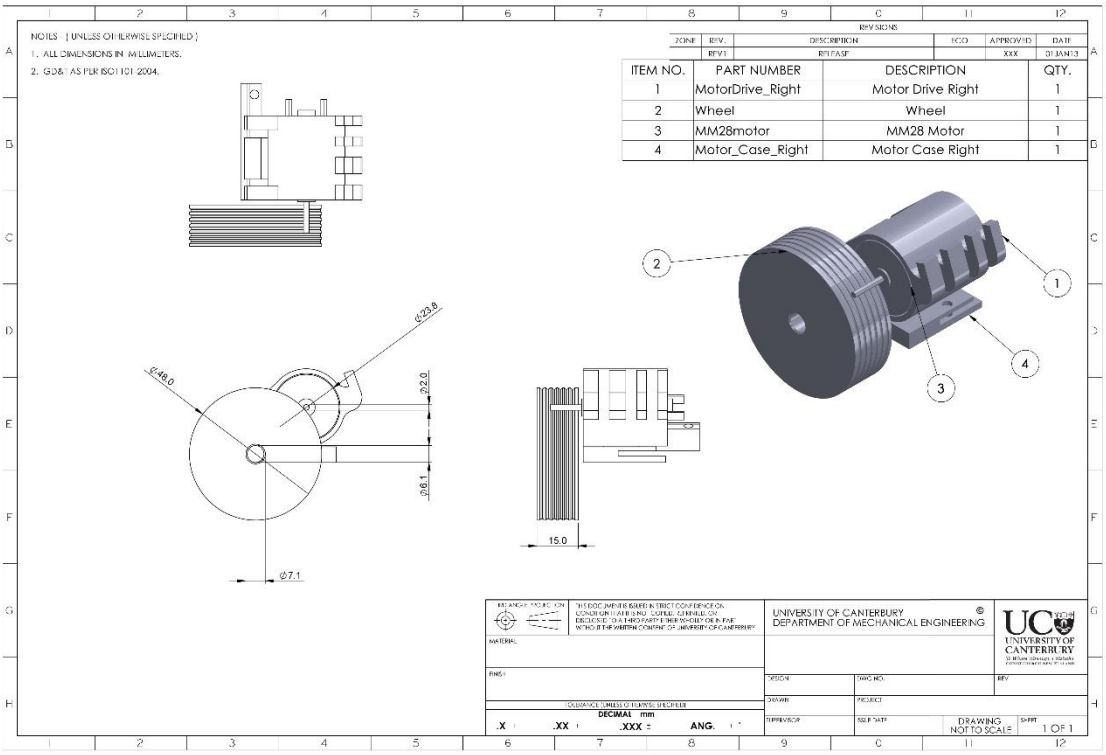


Figure 5: Final motor housing and wheel assembly drawing.

4.3.2 Wheel Traction iterations

Transmission from the motors to the wheel is determined by the static coefficient of friction and the normal force acting on the wheel. As mentioned previously the normal force was determined by the motor housing/tensioner, but the static coefficient can also be changed by changing the contact area as well as the contact surfaces.

From consultation with other group and 3rd year mechatronic students, the initial design for the wheel was made from PLA and the casted in high temperature RTV (Room Temperature Vulcanised) silicone. The silicone had sufficient traction on the course, the main issue was the motor shaft would eat into the silicone causing pieces to fall off causing inconsistent motion control. When larger wheels were incorporated, the decision was made not to continue with the silicone wheels due to no traction advantage compared to O-rings over the silicone. See Figure 6 and 7 for the wheel design iterations where it shows the silicone wheels then the smaller wheel with O-rings then the larger wheel with O-rings. The difference between the two O-rings designs is the previously mentioned change in size

and the number of O-rings was increased to provide a greater contact area with both the shaft and ground.

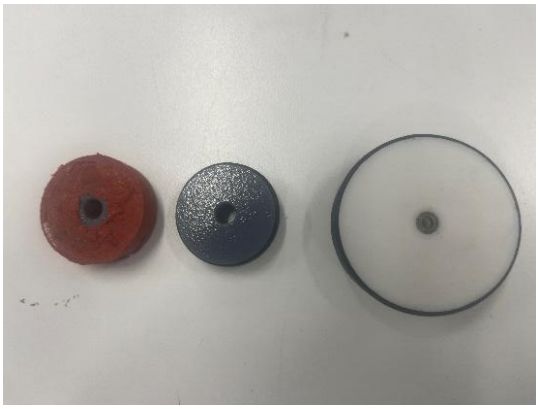


Figure 6: Design evolution of the wheels from start on the left to final on the right.

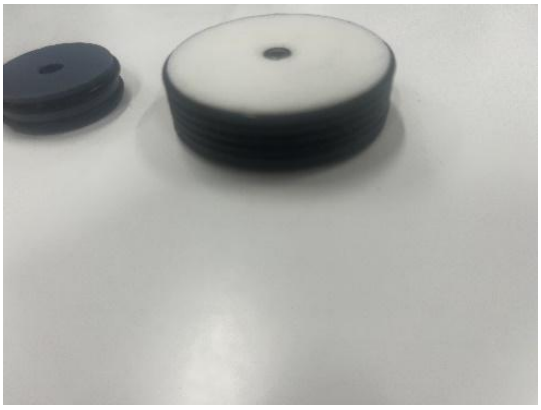


Figure 7: O-rings on large wheel to increase friction coefficient.

There was also slipping issues from just using the shaft which was very slick not allowing for much traction against the wheel. The first iteration was to use heat shrink on the motor shaft which would allow for better traction against the wheel. This was the design that was used through most of the testing, but it had the main drawback that only after a few attempts it would slip off the shaft. Superglue was tried to give the heat shrink more grip onto the shaft which did not improve the adhesion. As previously mentioned, 3D printed shaft expanders which were press fit onto the shaft which worked initially but then the shaft started to spin inside the expander for which was also tried to superglue to no avail. Finally, electrical tape was tried by wrapping it multiple times around the shaft which seemed to adhere to itself and becoming tighter under the tensioning force instead of slipping off which is why it was used in the final design.

Figure 8 shows the motor speed through a test run, this data showed that there was a consistent higher level of PWM values on the left motor throughout the course. From this the physical transmission was looked at and there was a difference in the gear ratio due to the amount of electrical tape around each shaft which was causing the left motor to have to run faster than the right. Based on this information equal lengths of tape were measured out and rapped around the shafts which improve the balance between the motors and led to the robot going straight more consistently.

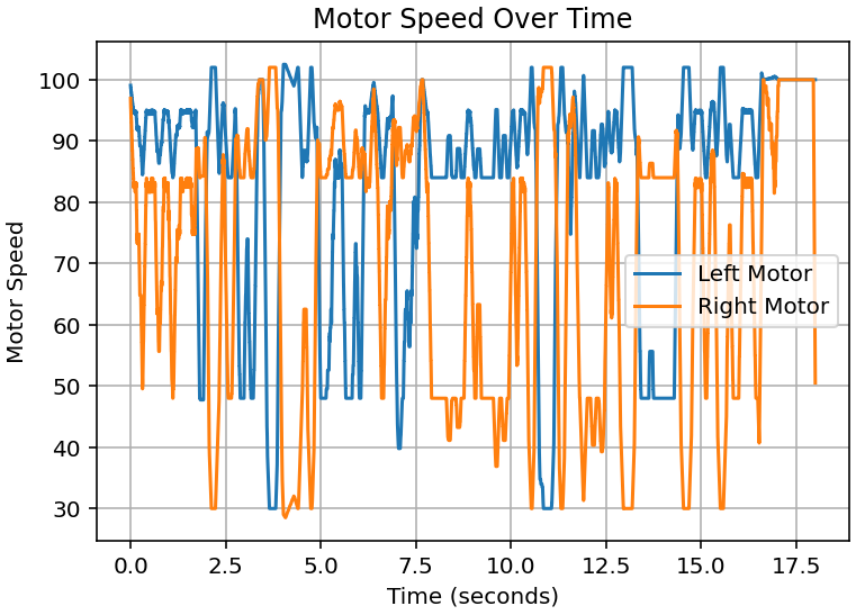


Figure 8: Motor speed (PWM) over time (seconds) around the test loop.

4.3.3 Battery mount and chassis iterations

The initial layout consideration was to have the batteries above the PCB base plate in a tower layout above the PCB between the two motors. This layout made debugging difficult as it was hard to work around the battery casing and it was hard to use the test points. To speed up testing and debugging the battery position was decided to be in between the microcontroller and the sensor board. This lowered the centre of mass however the main issue with this layout was that the inertia of the robot increased significantly as the batteries are the main weight component. This did not seem to have a significant effect on the cornering of the robot.

Another consideration to make is bringing the sensors in closer to the board. This would make the robot overall inertia lower due to decrease in length. When decreasing the length

of the robot during testing it was found to be more inconsistent during testing as the cornering improved significantly. There seemed to be a relationship between the sensor position and the wheelbase which allowed for effective cornering especially around the hairpin. Shown in Figure 9 is the final iteration of the chassis and sensor position which kept the knub in a similar position however moved the sensors much closer to the wheels. This allowed for the robot for the first time to travel around the track without leaving it.

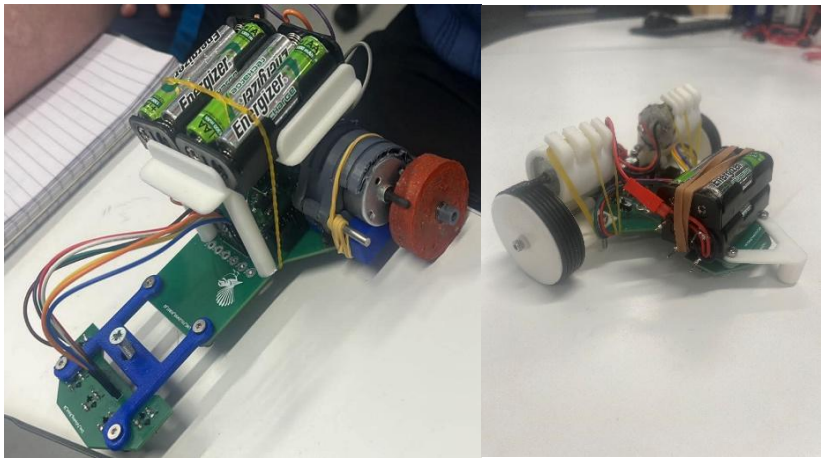


Figure 9: Final design layout with shorter overall distance (right) vs the original layout (left)

5 Electronics

5.1 PCB Design

The leading idea for the circuit board is design flexibility. It will incorporate a microcontroller, motor control circuitry, sensors, and supporting components required for the robot to operate. Each component will need to be arranged and traced strategically to avoid electromagnetic interference (EMI) between signals. The PCB will follow the design constraints outlined in the design specifications section.

5.1.1 PCB board layout

This project's PCB layout uses two separate PCBs: a main PCB and a secondary PCB. The secondary PCB will host the sensor array and other sensor-related components, while the main PCB will host all other components. This arrangement allows for tuning to find the optimal sensor-to-microcontroller distance. The capacitor and resistors will be surface mount as opposed to through-hole, as this allows for a more minimalist design and reduces the overall weight of the board; it has the drawback of being more difficult to solder.

5.1.2 Sensor board

When designing the sensor board. Caution was taken to design two types of boards, one with the sensors equally spaced and one with them closer together. This was done due to the unknown behaviour of the sensors, and this covered the behaviour of super sensitive sensors and sensors that may be more unreactive.

5.1.3 Main board

When designing the main board all standard precautions were taken including make sure traces were the right width and all connectors were outside of the board. Multiple test points were included at crucial spots including but not limited to sensors, motor outputs, checking battery outputs.

5.2 PCB Iterations and troubleshooting issues

After initial PCB soldering. The test light on the PCB would illuminate through communication through the PICO Bluetooth bootloader. Unknown problems prevented communication over the serial wire through the Arduino bootloader. Through extensive testing, it was found that the RX pin was soldered insufficiently, and there was a gap between the board and the pin. Initial attempts to drive the motor failed because the

PCB layout routed the motor driver from a pin without PWM (pulse width modulation). This led us to solder a wire from that pin to a PWM pin. There was also an issue where, when we used the switch to turn the microcontroller off, the right motor would have full battery power flowing through it. After testing with an oscilloscope, it showed an unusual voltage on the motor signal; we replaced the MOSFET for the motor driver, which fixed the problem and was likely the issue all along.

During testing, it was found that when the motors tried to ramp up quickly to high speed, they drew a lot of current all at once. This causes the voltage to sag throughout the circuit. This caused the microchip to brown out. The capacitors compensate for this voltage drop and supply the motor with the required voltage. In later discussions, it was determined that the diode between the battery and the regulator supplying the microcontroller had a 0.7-volt drop, which could also have contributed to the chip browning out, but this would be hard to determine without proper testing.

Taking precautions to ensure soldering was done correctly. During debugging, we found that moving the motors caused the wire connections to wear out and fray, leading to inconsistent connections. We addressed this by soldering the connections thoroughly and then applying hot glue around them to prevent fraying.

5.3 Bluetooth Programming

Programming over Bluetooth was attempted during the initial programming sessions. This was done using a PICO microcontroller. The advantage of including wireless programming is the speed at which the constant values can change on the fly while testing the robot. Uploading the program to the microcontroller was very inconsistent due to weak connections or other issues that were unable to be determined. This caused programming times up to 15 minutes trying to debug what issues are causing it not to initiate the handshake.

6 Computation

The software was written in C++ using the Arduino framework. Shown in Figure 10 the program operates as a continuous control loop that reads five sensors, calibrates the readings, determines the line position, calculates motor correction, and updates the motor outputs.

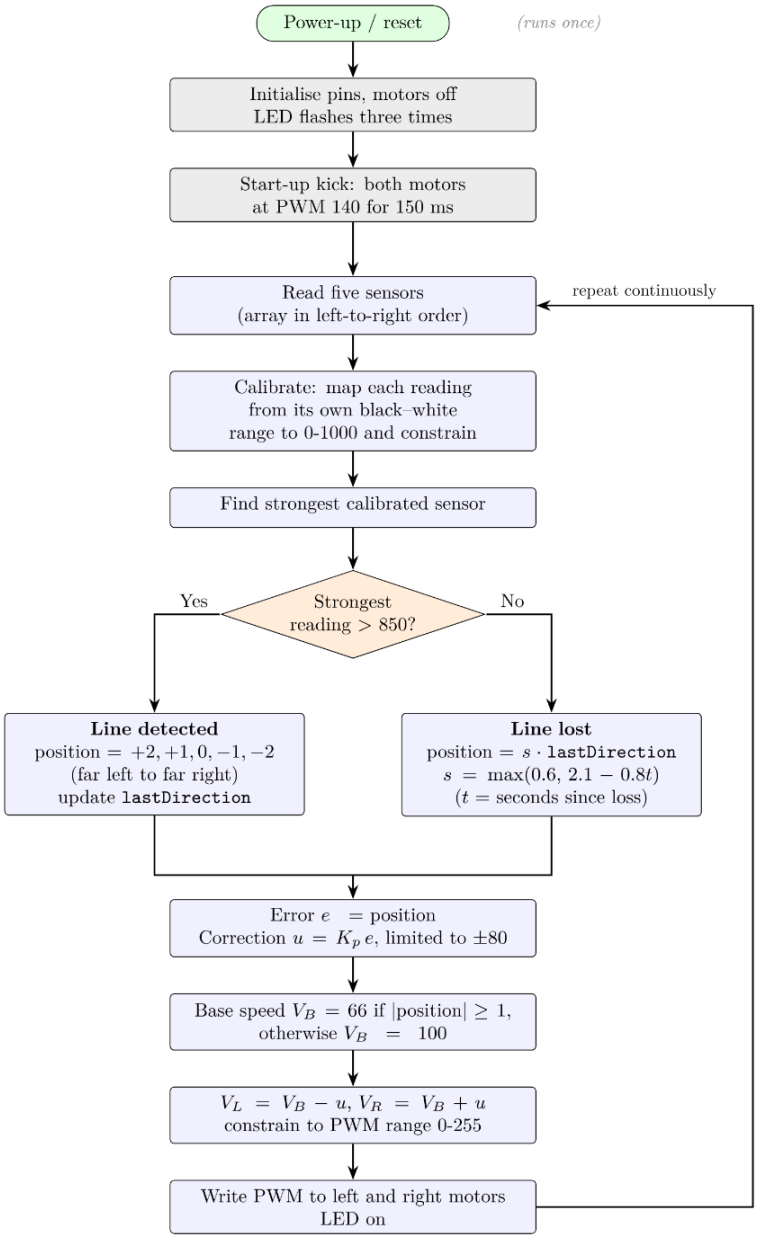


Figure 10: Software control loop of the line following robot. Start-up actions run once; feedback loop stages then repeat continuously.

6.1 Sensor Reading and Calibration

Five sensors are arranged across the front of the robot. They are read sequentially and stored in an array whose order matches the physical left-to-right arrangement.

During initial testing, the sensors produced different readings on the same surface, so raw readings could not be compared directly. This is due to the software seeing a phantom line when it is just sensor noise. Individual black and white reference values were therefore measured for each sensor. To prove our hypothesis, we recorded the values for each sensor over the same line, the results shown in Figure 11 proved that each sensor recorded a different value for the same input.



Figure 11: Graph demonstrating variances in readings for the same line (value 2 = sensor 1, value 6 = sensor 5)

To fix this issue each raw reading is mapped from its own black–white range onto a common range of 0 to 1000 and constrained to that range. This allows the sensors to be compared consistently and provides a common scale for the line-detection threshold. A trial using a single generic calibration for all sensors (white = 950, black = 350) was later abandoned in favour of the individual values demonstrated in Table 6 below.

Table 6: Calibration thresholds for sensor array

Sensor	1	2	3	4	5
White reading	967	967	969	971	972
Black reading	336	288	423	373	352

6.2 Line Position Detection

The line position is determined from the sensor with the largest calibrated reading. If that reading exceeds a threshold of 850, the line is considered detected and the sensor index is converted to a position:

Sensor	Far left	Left	Centre	Right	Far Right
Position	-2	-1	0	1	+2

The sign convention follows from the physical sensor arrangement and the motor implementation. This gives five discrete positions rather than a continuous estimate. The resolution is limited, but the method was simple, computationally efficient, and sufficient for the final implementation. The most recent valid position is stored and used to determine the search direction if the line is list.

6.3 Motor control

The final controller is proportional: $u(t) = k_p e(t)$ where e(t) is the line position and $K_p = 18$. The correction is limited to ±80 using contrain() and applied differentially. $V_L = V_B - u$, $V_R = V_B + u$, where V_B is the base speed. Motor commands are constrained to the PWM range of 0-255.

Speed control

The base speed depends on the line position. $V_B = 66$ if $|\text{position}| \geq 1$ and $V_B = 100$ if $|\text{position}| < 1$. The speed is reduced when the line is detected at the outer sensors, where larger corrections are required. Increasing speed improved straight-line performance but reduced reliability through sharper corners, so cornering reliability was prioritised. During line-loss recovery, the commanded position is at least 1 for approximately the first 1.4 s, so the reduced speed also applies early in a recovery.

Line Loss Recovery

On start-up, the LED flashes three times (500 ms on, 500 ms off), which indicates that start-up is completed and provides a delay of approximately 3 s before the robot moves. The delay was added to make sure the capacitors were charged before the initial current spike. Both motors are then driven at PWM 140 for 150 ms before being reduced to the base speed. This kick was intended to overcome static friction and start the motors reliably from start.

6.4 Development of the Control Algorithm

The software was developed over 19 saved versions (countless unsaved versions of just tweaks and testing). Table 5 shows the changes made in response to track testing.

Table 5: Development and tuning of controller parameters across successive iterations.

Version	Position method	K_p	K_d	Base speed	Line threshold
1	Hardware test (no controller)	-	-	PWM 50 fixed	-
2	Dominant raw sensor	10	10	50	-
3	Dominant raw sensor	10	10	50	-
4	Dominant raw sensor	10	10	30	-
5	Weighted average (raw)	40	45	30	-
6	Weighted average (calibrated)	30	2.5	35	-
7	Weighted average	40	0.03	100	720 (sum)
8	Weighted average	21.5	0.04	60	750 (sum)
9	Weighted average	18	0.02	75	750 (sum)
10	Weighted average	40	0.02	100	720 (sum)
11	Weighted average	40	0.03	121	850 (sum)
12	Weighted average	40	0.04	100	940 (sum)
13	Weighted average	60	0	100	850 (sum)
14	Weighted average	40	0	100	720 (max)
15	Weighted average	40	0	127	720 (max)
16	Strongest sensor	40	0	127	720
17	Strongest sensor (motor test)	40	0	225 (test)	800
18	Strongest sensor	22	0	80	600
19	Strongest sensor	18	0	100 / 66	850

Hardware verification: The first iteration was a hardware check rather than a controller. It read the five sensors, printed the values over the serial link, drove both motors at a fixed PWM of 50, and accepted serial commands for the motors and LED.

Initial controller: The next three iterations determined the line position from the dominant raw sensor, giving an error of e = 3 − dominant sensor and therefore five discrete error values. A proportional-derivative controller was used with $K_p = 10$ and $K_d = 10$. An integral variable was declared but never used, so there was no integral action. The derivative was updated at intervals of 500 loop iterations rather than on every loop. The base speed was 50 and later 30, and the motor outputs were limited to 100. A fault in which the stored maximum sensor reading was not reset between loops was corrected during this stage. A serial command interface for adjusting gains and speed was also included (see Bluetooth Control).

Weighted average and calibration: A weighted average of the five sensors (+2 to −2) was used to give a continuous line position. The first attempt used uncalibrated readings and contained a variable-scope error, so the position was never updated, and the method could not be evaluated. Per-sensor calibration was then introduced, together with lower limits on the calibrated values to prevent a zero denominator, and the derivative was calculated on every loop.

Time-based derivative, line-loss recovery, and speed control: The derivative was divided by the elapsed time measured with the microsecond timer. A line-detection threshold on the total calibrated reading, last-direction memory and line-loss recovery were added, together with a base-speed reduction proportional to line displacement. These changes addressed the robot losing the line completely and failing to recover.

Corner detection and braking: Two iterations trialled corner detection with hysteresis (entering a corner at |position| > 0.9 and leaving below 0.5), a reduced corner speed of 35, and a brief motor cut-off before corners. The first of these contained syntax errors and a redundant speed calculation that overrode the corner speed. In the second these were corrected and the braking time was set to zero, which disabled braking. This logic was not carried into later iterations.

Best observed performance: One iteration, using the weighted-average method with $K_p = 40$, $K_d = 0.02$, a base speed of 100 and a line threshold of 720, achieved a lap time of approximately 15 seconds. When it was tested later on, this performance could not be reproduced.

Attempts to recover performance: Several iterations kept the same or very similar proportional and derivative values while the base speed and line threshold were varied (base speed 121 then 100, threshold 850 then 940, K_d 0.03 to 0.04). A subsequent iteration trialled a single generic calibration for all sensors together with $K_p = 60$ and no derivative. The individual calibration and the earlier configuration were then restored with $K_d = 0$. A further iteration increased the base speed to 127, removed the speed reduction, and increased the line-loss search magnitude to 2.2. During this period, the code no longer behaved as expected, and the original 15-second version also failed to perform as it had previously.

Strongest-sensor method: The code was restructured so that the line position was taken from the strongest calibrated sensor only. This simplified the position calculation and produced more repeatable behaviour.

Motor startup investigation: One iteration replaced the control loop with a full-power motor test to examine motor behaviour. Line following was then restored with $K_p = 22$, a base speed of 80, a threshold of 600 and a gradual start-up ramp.

Final implementation: The final version uses the strongest-sensor method with $K_p = 18$ and no derivative term. The line threshold is 850, and the base speed is 100 on straighter sections and 66 when the line is at the outer sensors. Line-loss recovery reduces the commanded search position from 2.1 to 0.6 over approximately 1.9 seconds, and the ramp was replaced by a start-up kick of PWM 140 for 150 ms.

Iteration of the Derivative Term

No integral term was used at any stage. The derivative term was tested extensively, but the robot performed better with proportional control alone. The plan was to tune K_p on its own first and then add K_d to damp the response. Adding K_d did not give a consistent improvement, and the following were tested:

- K_d values close to K_p ($K_p = 40$, $K_d = 45$).
- K_d greater than K_p, and K_d less than K_p.
- K_p varied with K_d held constant.
- Very low K_d values ($K_d \leq 0.04$).

The base speed was adjusted throughout to see whether it improved cornering and line retention.

The scale of K_d changed during development. In the early iterations the derivative was the change in error per loop (or per 500 loops), so gains of 10 to 45 were used. After the derivative was divided by elapsed time in seconds, comparable gains were 0.02 to 0.04.

The 15-second lap was recorded with a small derivative gain ($K_d = 0.02$). The loss of that performance occurred both with the derivative retained ($K_d = 0.03$ to 0.04) and with it removed ($K_d = 0$), so it was not attributed to the derivative term. Derivative control was removed from the final controller because proportional control alone gave the best observed performance and the derivative added complexity without a demonstrated benefit.

Bluetooth Control

A wireless tuning and monitoring interface was developed to allow gains and speed to be adjusted during track testing without reprogramming. A Raspberry Pi Pico 2W running MicroPython was configured as a Bluetooth Low Energy to UART bridge. It implemented an HM-10-style service (UUID 0xFFE0, characteristic 0xFFE1), forwarded commands from a phone to the robot's microcontroller at 115200 baud, and returned serial output as BLE notifications.

Four program versions were produced:

1. A basic LED blink program to verify the Pico and its toolchain.
2. A first bridge implementation forwarding data to the robot, returning serial output, and providing LED blinking status.
3. An improved bridge with write-without-response support, larger receive buffers, received-text trimming, automatic newline appending to match the robot's line-based parser, serial debugging output, connection status indication and error handling.
4. A further saved copy of the improved bridge.

The robot firmware in several early iterations included a command parser for K_p, K_d, base speed, LED control and start/stop. Despite many hours of work, the link could not be made to operate reliably. Because it was delaying progress on line following, it was abandoned in the early-to-middle stage of the project, and all later tuning was done by editing parameters and reprogramming the board.

Controller Tuning

Tuning was performed experimentally on the track. The main trade-off was between speed and cornering reliability. Higher base speeds improved straight sections but made sharper corners harder to follow, so speed is reduced when larger steering corrections are needed. The final values ($K_p = 18$, correction limit ± 80, base speeds of 100 and 66, threshold of 850) were selected from observed track performance.

Microcontroller Considerations

The final software uses millis() to time, line-loss recovery, and delay() at start-up, and earlier iterations used micros() for the derivative. The accuracy of these functions, and the PWM frequency produced by analogWrite(), depend on the microcontroller clock. During testing a clock frequency of 8MHz was found have the best results. A higher frequency resulted in slower behaviour. The calibration values are hardware-dependent, so changes in sensor characteristics, sensor positioning or track lighting would require them to be measured again. Motors can draw substantially more current under load or stall than in normal running. The motor driver and supply must therefore handle the peak current, particularly at start-up and in corners.

7 Comparison between other groups

The performance of the robot was compared with two other groups using different motors and drivetrain configurations. The first group (Group 03 G) used friction wheels with an MM10 motor, while the second group (Group 03 K) used 3D-printed spur gears with an MM10 motor.

Group 03 G provided the closest comparison to this design because they used friction wheels. They found that positioning the sensor board too far from the wheels caused the robot to detect the adjacent line during U-turn bends. They experienced a sensor-positioning issue where the rear weight distribution lifted the sensors and changed ground clearance. Both groups required accurate sensor positioning to maintain reliable line detection.

Wheel configuration was also an area of development for Group 03 G. They tested multiple wheel configurations by varying the number of O-rings to achieve suitable performance. Similarly, they replaced the silicone wheels initially used with O-rings to improve consistency in wheel-to-track contact. Group 03 G also found that precise tensioning was important for their tuning, while their design required additional capacitors to reduce the risk of motor burnout.

Group 03 K used spur gear transmission with an MM10 motor. They initially used friction wheels but switched to a 3D-printed gear drivetrain after the original design proved ineffective. After switching to a gear transmission, they still faced inconsistent performance, with wheels occasionally failing to turn. They also added a third diode to help prevent brownout. Despite these changes and iterations, the robot still couldn't complete the entire track, showing that changing the drivetrain alone wasn't enough to achieve consistent performance.

8 Results

8.1 Performance on the demonstration course

The robot completed all three runs without losing track of the line. Table 7 summarises its performance across the three attempts. The first run used a conservative PWM value of 80, allowing the robot to follow the line reliably without noticeable deviation. In the second run, the aim was to increase the speed along straight sections of the course. The program increased the PWM by 20 when the motor values matched, allowing the robot to travel faster when moving straight. Although this approach worked well during testing, the robot appeared to stagger on the actual course whenever it attempted to accelerate. This was likely due to slight differences in motor performance, which may have caused the robot to repeatedly switch between straight-line movement and position correction when an inner sensor detected the line. These frequent adjustments reduced its overall speed, resulting in a slower time than the first run. In the final run, the straight-line speed adjustment was removed, and the PWM was increased to 100. This simplified approach produced the fastest time of the three runs, suggesting that a consistently higher speed was more effective than the additional speed-control logic. In retrospect, testing three programs with progressively higher speed settings and different reliability levels would have provided a clearer comparison of the speed-consistency trade-off and helped identify the most effective configuration before the final run.

Table 7: Lap time results across three runs.

Run	Outcome	Lap time (s)
1	Completed	18.77
2	Completed	19.11
3	Completed	17.68
Best		17.68
Average		18.52

9 Reflection

This section reflects on the team's process, dynamics, and key learnings over the course of the project. Overall, the team collaborated well and made design decisions collectively, drawing on each member's individual work and input. However, the project also faced several setbacks, including hardware failures from over-testing near the deadline and a motion-transmission design that was not established early enough, which significantly hindered later debugging. The following sections evaluate these successes and setbacks in detail.

The initial project plan was to ensure the PCB was fully operational, driving the motors and successfully sending and receiving serial data, before progressing further. Ordering the PCB ourselves rather than through the university helped accelerate the group's progress. The PCB was fully soldered before some other groups had even received their boards. This proved to be a major advantage, and we would recommend this approach to future groups.

In hindsight, the initial motion transmission made debugging and testing considerably more difficult. Not being able to determine whether the code or mechanical slippage caused a failure led the group to oscillate between adjusting the code and making minor transmission changes, without isolating the single variable responsible. This highlights the importance of securing a reliable motion transmission early in the project. Designing and iterating in CAD was more tedious and time-consuming than debugging electronics and code, especially given how effectively LLMs can assist with code debugging compared to the more limited integration of AI tools into the SolidWorks workflow.

Team dynamics played an important role in the robot's progress, and the group worked collaboratively throughout the project. Although some members devoted more time to the robot than others, particularly during intensive periods of development, communication was maintained throughout. The code was kept organised and in versions whenever there was a change in code, changes were communicated between team members, and questions were asked whenever clarifications were needed. This allowed the team to coordinate work around individual availability while keeping everyone informed of the robot's progress. The robot would not have reached its level without the combined effort and contributions of the team.

As the demonstration deadline approached, extensive testing led to hardware fatigue and became a major issue. For example, the best time recorded on 3rd October was 15 seconds

around the track. On the night before the demonstration, the robot began behaving erratically for unknown reasons, with initial assumptions of worn motor brushes or MOSFET damage due to high capacitance. This experience showed the importance of limiting further testing once the robot performs consistently and meets the required objectives, unless testing is guided by a clear plan for improvement.

If the project were undertaken again, the overall design of the robot would remain similar, as the existing design provided a reasonable balance between compactness and space required for the battery holder and separate sensor board. One area for improvement would be the motion transmission. Laser-cut acrylic gears could provide a more consistent speed ratio than friction wheels, which are simpler to implement but can be affected by mechanical slip. The results of the top-performing teams suggest that performance depended on several aspects of the design, including mechanical structure, transmission, and code. Future improvements should therefore focus on the overall integration of the robot's mechanical and electronic systems rather than assuming that changing the transmission alone would lead to better results.

10 Conclusion

The line following robot was designed, built, and successfully demonstrated, completing the course with a best lap time of 17.68 s.

The key outcomes were:

- The ATmezga4808-AFR microcontroller was selected over the ATmega328-PU because it offers a smaller SMD package, remappable peripherals, and more SRAM for Bluetooth buffering.
- The MM28 motor was selected for its higher stall torque and efficiency, especially for the friction wheel transmission.
- After comparative analysis against belt-and-pulley and spur-gear alternatives, the friction wheel direct drive was selected as the transmission, scoring highest on the weighted decision matrix (4.45/5.00).
- Iterating the wheel diameter from 33mm to 49mm resolved the issue of the robot not having enough torque.

The primary area for future improvement is removing the diode between the regulator and the battery, as its 0.7 V drop caused the microcontroller to brown out when the motors drew too much current. This improvement carries inherent risks, as the diode protected the rest of the circuit if batteries are inserted incorrectly.

11 References

[1] ENMT221, "Individual and Team Line-Following Robot Brief", Semester 2, 2026, Version 2.1.

[2] Microchip Technology Inc., *ATmega4808/4809 Data Sheet*, DS40002173C, 2021.

[3] Microchip Technology Inc., *ATmega328 Family Data Sheet*, DS40002061B, 2020.

[4] ENMT221, "MCU Section Layout Guide", companion guide, Rev r2, 2026.

[5] Multicomp, *Miniature Motors Datasheet*, Model MM10 / MM28, Rev. V1.1, Jun. 22, 2012. [Online]. Available: https://www.farnell.com

[6] Vishay Semiconductors, TCRT5000 Reflective Optical Sensor with Transistor Output, document 83760, revision 1.7. https://www.vishay.com/docs/83760/tcrt5000.pdf

12 Appendix

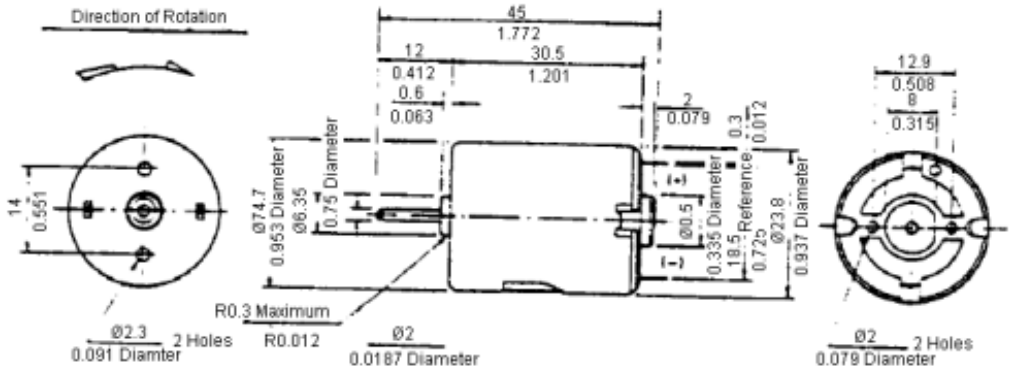
12.1 Appendix A

Miniature Motors

multicomp

Model MM28





Dimensions : Millimetres (Inches)

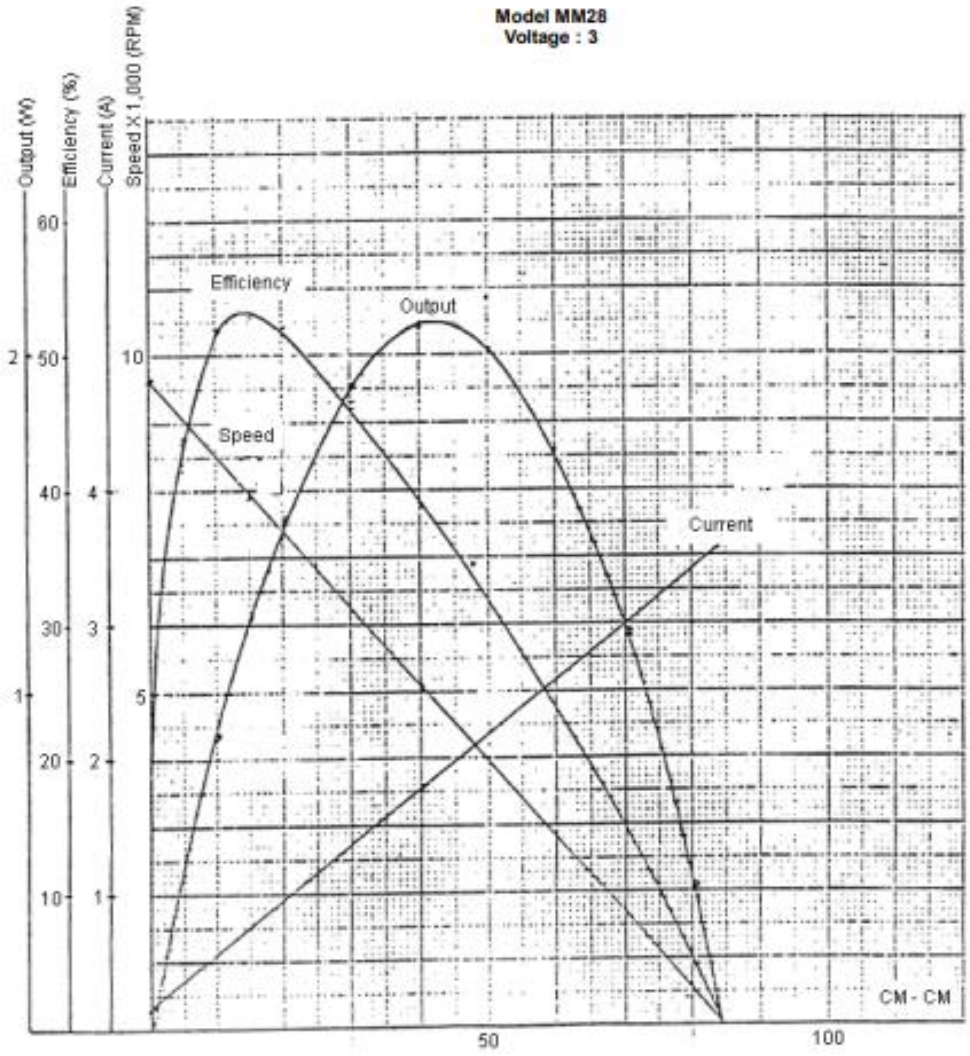
Specification Table

Model	Operating Range	Approximately Weight
MM28	3 - 6 V	42 g

Nominal	No Load		At Maximum Efficiency					Stall Torque g-cm
Constant V	Speed rpm	Current A	Speed rpm	Current A	Torque g-cm	Output W	Efficiency %	
3 V	9,600	0.22	8,000	0.99	20	1.64	55.2	112

Miniature Motors

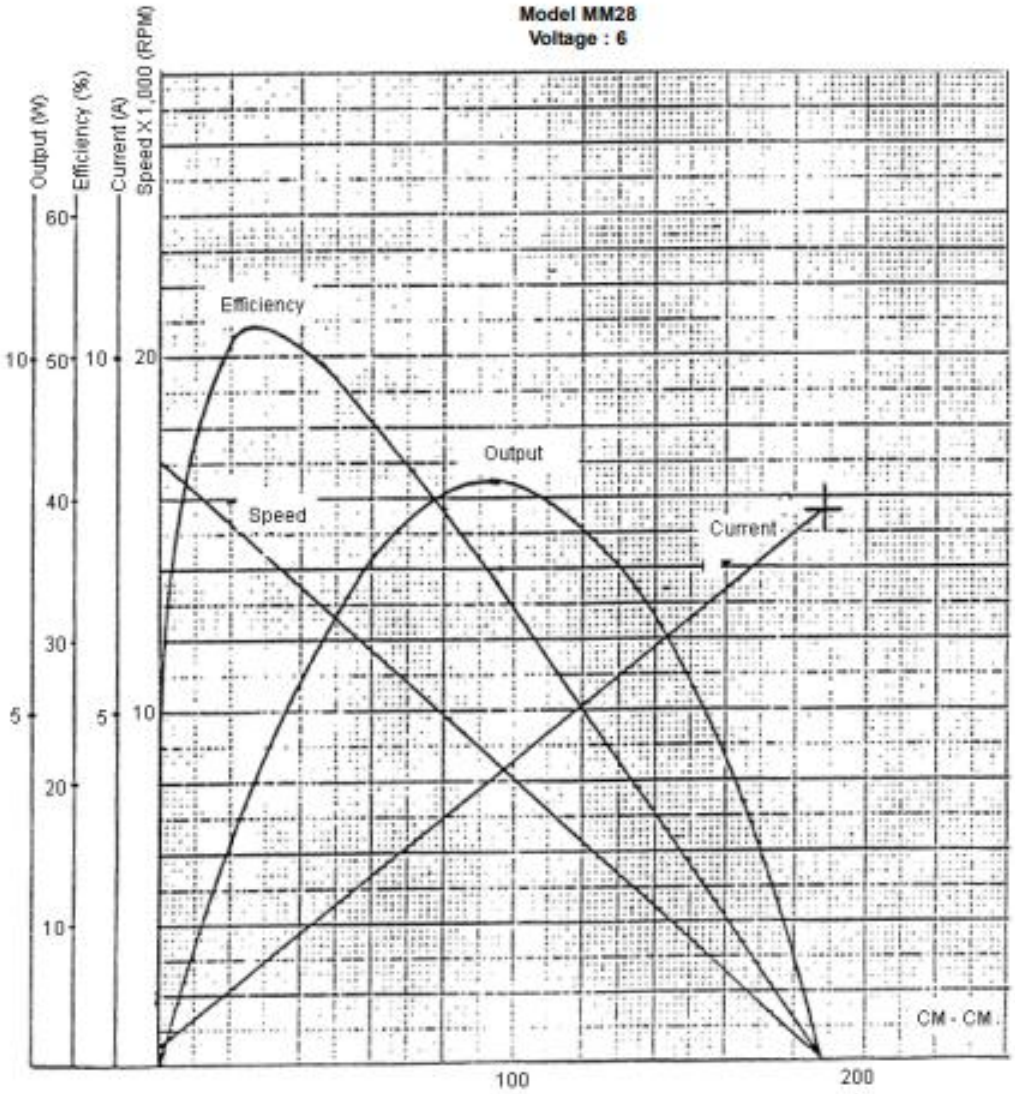
multicomp



Description	Current A	Speed RPM	Torque CM - CM	Output W	Efficiency %
No Load	0.15	9,638	-	-	-
Maximum Efficiency	0.76	7,926	15	1.22	53.3
Maximum Output	1.78	5,072	40	2.08	38.9

Miniature Motors

multicomp



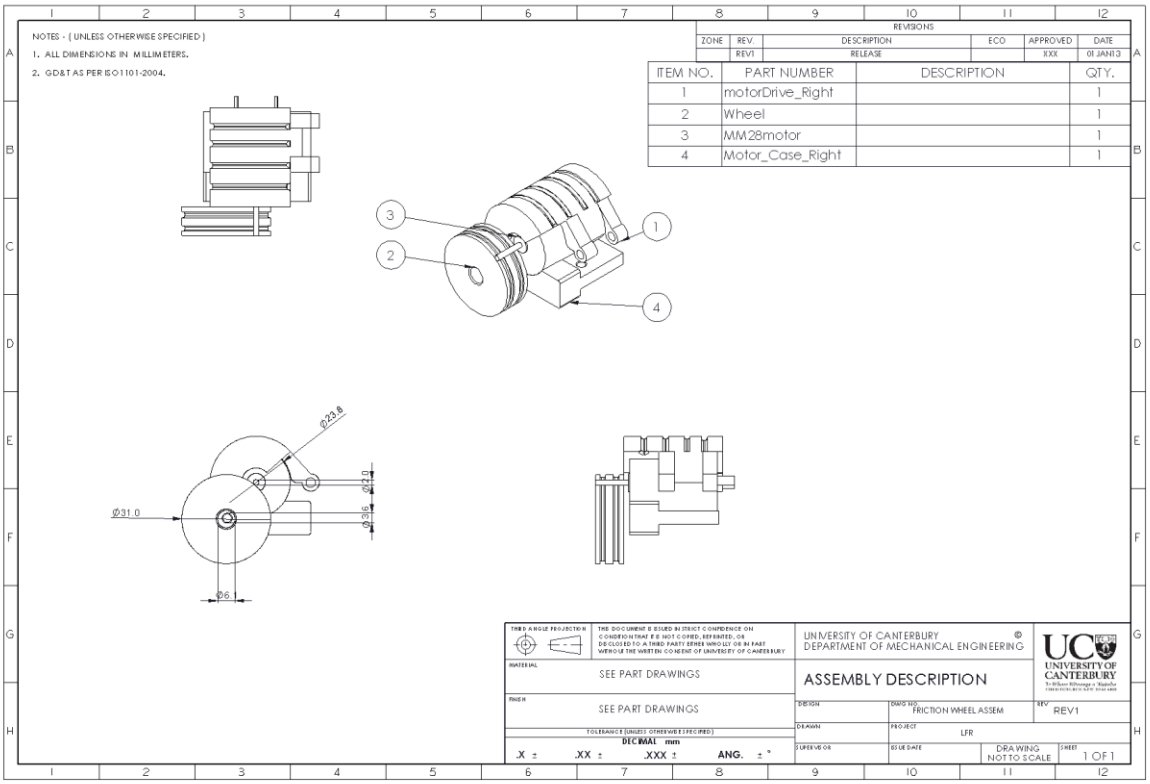
Description	Current A	Speed RPM	Torque CM - CM	Output W	Efficiency %
No Load	0.21	17,100	-	-	-
Maximum Efficiency	1.28	14,689	26.52	4	52
Maximum Output	4	8,550	94.06	8.25	34.3

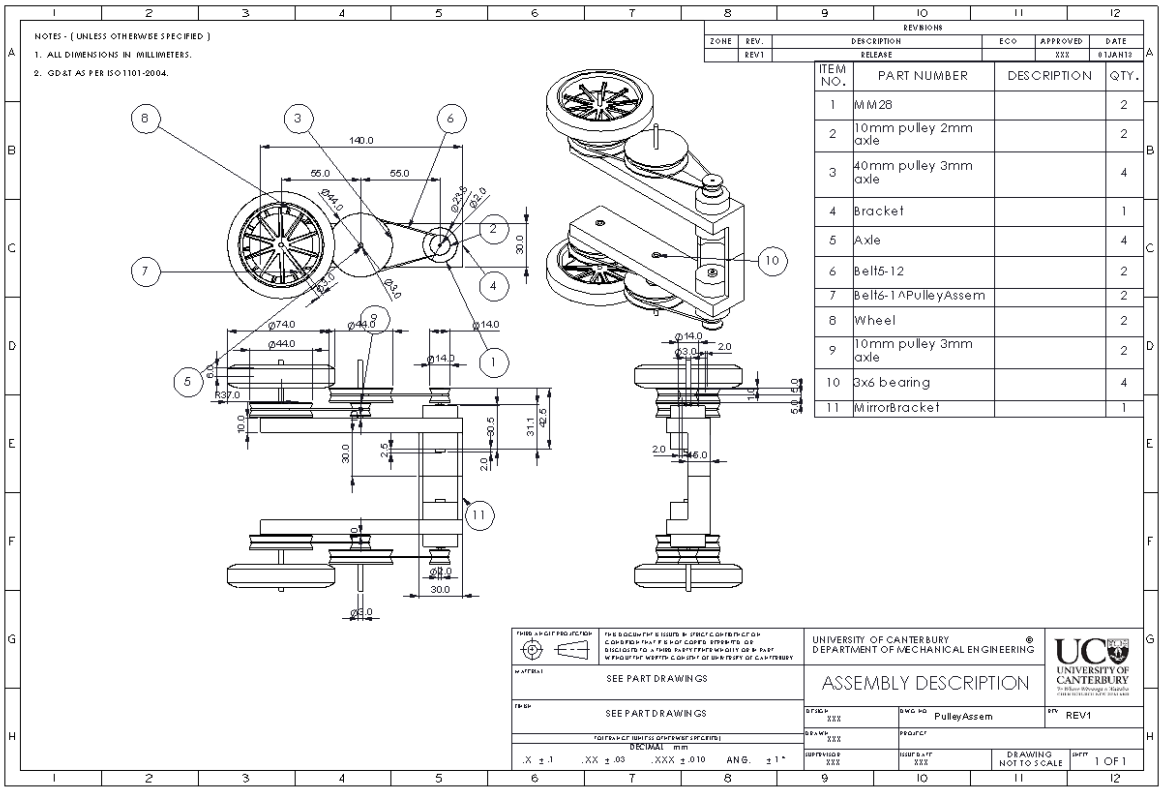
Important Notice : This data sheet and its contents (the "Information") belong to the members of the Premier Farnell group of companies (the "Group") or are licensed to it. No licence is granted for the use of it other than for information purposes in connection with the products to which it relates. No licence of any intellectual property rights is granted. The Information is subject to change without notice and replaces all data sheets previously supplied. The Information supplied is believed to be accurate but the Group assumes no responsibility for its accuracy or completeness, any error in or omission from it or for any use made of it. Users of this data sheet should check for themselves the Information and the suitability of the products for their purpose and not make any assumptions based on information included or omitted. Liability for loss or damage resulting from any reliance on the Information or use of it (including liability resulting from negligence or where the Group was aware of the possibility of such loss or damage arising) is excluded. This will not operate to limit or restrict the Group's liability for death or personal injury resulting from its negligence. Multicomp is the registered trademark of the Group. © Premier Farnell plc 2012

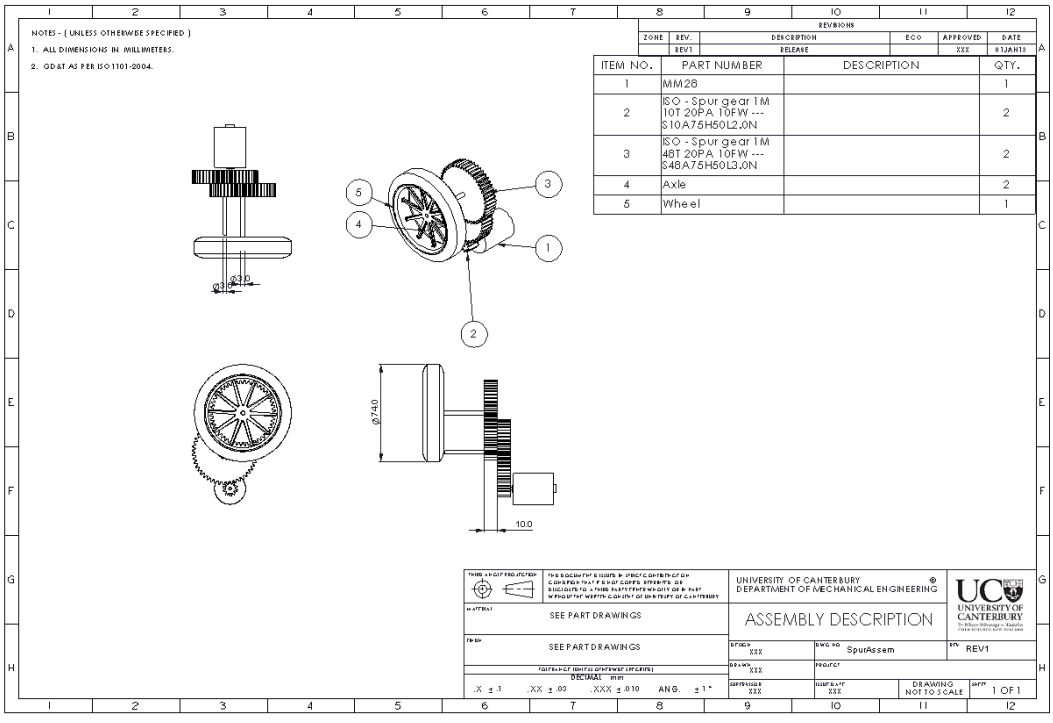
www.element14.com
www.farnell.com
www.newark.com

multicomp

12.2 Appendix B




NOTES - (UNLESS OTHERWISE SPECIFIED)
1. ALL DIMENSIONS IN MILLIMETERS.
2. GD&T AS PER ISO 1101-2004.
REVISIONS
ZONE | REV. | DESCRIPTION | ECO | APPROVED | DATE
REV1 | RELEASE | XXX | 01JAN13
ITEM NO. | PART NUMBER | DESCRIPTION | QTY.
1 | MM28 | 2
2 | 10mm pulley 2mm axle | 2
3 | 40mm pulley 3mm axle | 4
4 | Bracket | 1
5 | Axle | 4
6 | Belt5-12 | 2
7 | Belt6-1APulleyAssem | 2
8 | Wheel | 2
9 | 10mm pulley 3mm axle | 2
10 | 3x6 bearing | 4
11 | MirrorBracket | 1
140.0
55.0
55.0
Ø44.0
Ø23.8
Ø3.0
30.0
Ø5.0
Ø74.0
Ø14.0
Ø37.0
10.0
30.5
31.1
42.5
30.0
2.5
20.0
Ø8.0
Ø3.0
2.0
5.0
UNIVERSITY OF CANTERBURY
DEPARTMENT OF MECHANICAL ENGINEERING
SEE PART DRAWINGS
ASSEMBLY DESCRIPTION
PulleyAssem
REV1
1 OF 1


NOTES - (UNLESS OTHERWISE SPECIFIED)
1. ALL DIMENSIONS IN MILLIMETERS.
2. GD&T AS PER ISO 1101-2004.
REVISIONS
ZONE | REV. | DESCRIPTION | ECO | APPROVED | DATE
REV1 | RELEASE | XXX | 01JAN13
ITEM NO. | PART NUMBER | DESCRIPTION | QTY.
1 | MM28 | 1
2 | ISO - Spur gear 1M 10T 20PA 10FW --- S10A75H50L2.0N | 2
3 | ISO - Spur gear 1M 48T 20PA 10FW --- S48A75H50L3.0N | 2
4 | Axle | 2
5 | Wheel | 1
Ø3.0
Ø74.0
10.0
UNIVERSITY OF CANTERBURY
DEPARTMENT OF MECHANICAL ENGINEERING
SEE PART DRAWINGS
ASSEMBLY DESCRIPTION
SpurAssem
REV1
1 OF 1